

AWS Well-Architected Framework Review: ROCA Scripture and Lectionary Management Flow (Batch 4 of 4) with Cross-Batch Remediation Roadmap

1 Executive Summary

The Batch 4 Well-Architected review of ROCA's Scripture and Lectionary Management flow reveals a system that is functionally rich — spanning 29 API endpoints, 5 lectionary systems, 7 Bible translations, and offline-first mobile support — but architecturally exposed by six critical gaps that have persisted unresolved across all four review batches. Most urgently, missing Dead Letter Queues on async Lambda invocations risk silent data loss across reading plan updates and devotional generation [1]; absent WAF and GuardDuty leave public-facing scripture search endpoints vulnerable to injection attacks [2]; and JWT rotation without PKCE remains the platform's single most persistent security vulnerability, affecting every endpoint across all 8 ROCA flows [3]. These three systemic gaps compound with flow-specific risks: the ESV API's 5,000-request daily cap creates a hard third-party dependency that lacks a formal circuit breaker [4, 5], and the last-write-wins offline sync strategy will silently overwrite concurrent lectionary edits from multiple devices — a conflict best resolved by migrating to AWS AppSync with Optimistic Concurrency and Delta Sync [6, 7].

The consolidated cross-batch analysis estimates 240–280 total engineering days of remediation across all 8 ROCA flows. The most impactful action is a platform-wide Phase 1 sprint (Weeks 1–4) deploying WAF/GuardDuty, configuring DLQs on every async Lambda, and implementing JWT rotation — three force-multiplier fixes that simultaneously reduce risk across all flows. Batch 4 adds approximately 30–40 days of flow-specific work, prioritizing the Bible API circuit breaker via Lambda Powertools [8], GIN index creation on tsvector columns for up to 5,000x search improvement [9], and RDS Proxy deployment to prevent connection exhaustion from 29 concurrent Lambda functions [10]. Addressing these items in the recommended four-phase sequence ensures the highest-risk, broadest-impact deficiencies are resolved first while building the architectural foundations — Step Functions, EventBridge, AppSync — needed for long-term platform resilience.

2 Introduction and Architecture Overview

This report presents the fourth and final batch of the AWS Well-Architected Framework Review for the ROCA church management platform, focusing on the Scripture and Lectionary Management flow. This flow serves as the spiritual backbone of the platform, connecting the liturgical calendar, event scheduling, devotional content, and notification systems to the church year. It is the most theologically complex flow in the ROCA ecosystem, requiring precise alignment between ancient liturgical traditions and modern cloud architecture.

The Scripture and Lectionary Management flow encompasses 29 API endpoints spanning lectionary configuration, readings retrieval, calendar generation, sermon series management, scripture search, reading plans, bookmarks, devotional generation, preaching history, and PDF export [4]. The data layer consists of 8 PostgreSQL tables — `lectionary_config`, `lectionary_readings`, `scripture_assignments`, `sermon_series`, `reading_plans`, `reading_plan_progress`, `scripture_bookmarks`, and `preaching_history` — each with specialized column structures including JSONB fields for flexible translation and assignment storage [4].

The flow supports five distinct lectionary systems — Revised Common Lectionary (RCL), Roman Catholic Lectionary, Book of Common Prayer, Custom, and Freeform — each with its own cycle mechanics [4]. The RCL and BCP systems follow a 3-year A/B/C cycle that advances at the start of Advent rather than January 1, computed via a cycle-year algorithm that checks whether the current date falls on or after Advent Sunday to determine the effective liturgical year [4]. Easter dates are computed using the Anonymous Gregorian Method, validated for years 2000–2100 [4].

Scripture text delivery relies on the ESV API as the primary source (with a 5,000 requests/day limit) and Bible.api for additional translations, with Redis caching at a 24-hour TTL per verse and a local KJV fallback for quota exhaustion scenarios [4]. Audio playback uses HLS streaming from the ESV Audio API with AWS Polly text-to-speech as a fallback for non-ESV translations [4]. Full-text search is powered by PostgreSQL tsvector indexes targeting sub-300ms p95 response times [4]. The frontend comprises 16 components across React TypeScript web and React Native mobile, including 4 web pages, 3 mobile screens, 3 widgets, 4 shared components, and 2 modals [4]. Offline support uses IndexedDB via Dexie.js for web and SQLite for mobile, with a 7-day scripture pre-cache, a 50MB storage budget, and last-write-wins conflict resolution with delta sync on reconnection [4].

This Batch 4 review evaluates the flow across all six Well-Architected pillars, assesses seven new risks unique to this flow, cross-references unresolved gaps from Batches 1–3, and delivers a consolidated remediation roadmap covering all 8 ROCA flows.

3 Operational Excellence and Monitoring Architecture

Observability Gaps. The current architecture lacks distributed tracing across its 29 Lambda-backed API endpoints. AWS X-Ray should be enabled on all functions and API Gateway stages to provide end-to-end visibility into request flows, particularly for multi-step operations such as devotional generation and reading plan progress updates [11]. Without X-Ray, diagnosing latency spikes in the scripture delivery chain — which traverses API Gateway, Lambda, Redis, ESV API, and potentially Polly — requires manual log correlation across multiple services.

Runbook Deficiencies. No operational runbooks exist for two critical failure scenarios unique to this flow. First, the Easter Computus algorithm, while mathematically sound for 2000–2100 [4], has no documented procedure for validating outputs against known Easter dates or for handling years outside the validated range. Second, there is no runbook for ESV API quota exhaustion or outage, which would cascade into degraded scripture delivery across all active reading plans and Sunday readings. AWS best practices call for automated runbooks covering common failure scenarios in serverless architectures [11].

Infrastructure as Code. Lectionary configuration management — including the mapping of 5 lectionary systems, cycle year rules, and reading slot assignments — appears to be managed through application-level CRUD operations rather than versioned IaC templates. This creates drift risk when deploying across environments and makes it difficult to audit changes to liturgical data that affects the entire congregation's worship experience.

4 Security and Authentication Framework

Inherited Critical Gaps. Three security deficiencies identified in Batches 1–3 carry forward with undiminished severity. JWT rotation and PKCE remain absent, a finding that has persisted as Critical across all four batches [3]. AWS WAF is not attached to the API Gateway stages protecting the 29 endpoints, leaving them exposed to SQL injection and XSS attacks — a gap that AWS documentation explicitly warns against for REST APIs [2]. GuardDuty is not enabled for continuous threat monitoring of Lambda and API Gateway activity [2]. These gaps were escalated from High to Critical in Batch 2 when payment flows were assessed and remain Critical here because the scripture endpoints are publicly accessible [3].

Bible API Key Management. The ESV API and Bible.api credentials require rotation management through AWS Secrets Manager. Currently, API keys for third-party scripture providers are likely stored in environment variables or configuration files, which violates the Well-Architected Security pillar's guidance on secrets management. Automated rotation schedules should be configured to rotate keys at least every 90 days.

Multi-Tenant Lectionary Isolation. The `lectionary_config` table stores denomination-specific configurations including custom lectionary systems [4]. Row-level security policies in PostgreSQL should enforce tenant isolation so that one church's custom lectionary overrides cannot be read or modified by another tenant. The `lectionary_readings` table's override tracking mechanism must also enforce tenant-scoped access to prevent cross-contamination of liturgical data.

Rate-Limiting Protection. WAF rate-limiting rules should be applied per-endpoint, with particular attention to the scripture search endpoint (which accepts arbitrary user input for tsvector queries) and the devotional generation endpoint (which may invoke compute-intensive AI operations) [2].

5 Reliability and Fault Tolerance

Missing Dead Letter Queues. Async Lambda invocations across the flow — including reading plan progress updates, devotional generation, and preaching history logging — lack DLQ configurations. The Well-Architected Serverless Lens explicitly requires DLQs on every async Lambda invocation, with configured Maximum Retry Attempts and Maximum Record Age to prevent silent data loss [1]. This is a Critical finding that has persisted across all four batches [3].

Bible API Circuit Breaker. The ESV API's 5,000 requests/day limit [4] creates a hard dependency that can fail via HTTP 429 throttling or outright unavailability. The current fallback to cached/local KJV with exponential backoff is a reasonable degradation strategy, but it lacks a formal circuit breaker implementation. AWS Prescriptive Guidance recommends the circuit breaker pattern — tracking closed, open, and half-open states — to prevent cascading failures when a downstream service is unhealthy [5]. Reference implementations exist using Step Functions with DynamoDB [12] and Lambda Powertools [8]. The circuit breaker should track failure rates per Bible API provider independently, opening the circuit after a configurable threshold (e.g., 5 consecutive failures or a 50% failure rate over 60 seconds) and routing all traffic to the Redis cache and KJV fallback during the open state.

Offline-Online Sync Conflicts. The current last-write-wins conflict resolution strategy [4] is inherently lossy. Consider a scenario where a pastor overrides a lectionary reading on their laptop while offline, and a church adminis-

trator makes a different override on the web app — the last device to sync silently overwrites the other's change. AWS AppSync provides three superior conflict resolution strategies: Optimistic Concurrency (rejects stale mutations, returning the server item for client retry), Automerge (automatically merges non-conflicting fields), and Custom Lambda resolvers for domain-specific logic [6]. Combined with Amplify DataStore, AppSync supports offline data access with automatic conflict resolution on reconnection [13]. Delta Sync further optimizes the 7-day pre-cache strategy by allowing clients to hydrate their local cache with a base query and then receive only incremental updates, reducing bandwidth consumption for devices that frequently transition between online and offline states [7].

Reading Plan Streak Integrity. The 48-hour grace period for streak tracking [4] introduces edge cases around timezone boundaries and device clock skew. If `last_read_at` is stored in UTC but the grace period is evaluated against a user's local time, a user near the boundary could lose their streak due to timezone arithmetic errors. Streak updates should use atomic database operations (e.g., `UPDATE ... SET streak_count = streak_count + 1 WHERE last_read_at > NOW() - INTERVAL '48 hours'`) to prevent race conditions from concurrent devices, and the grace period logic should be evaluated server-side in a single timezone-aware transaction.

Saga Pattern for Reading Plans. Multi-step reading plan operations — such as creating a plan, generating daily assignments, and initializing progress tracking — lack compensating transactions. If assignment generation fails mid-way through a 365-day plan, there is no rollback mechanism. Step Functions should orchestrate these operations with compensating Lambda functions for each forward step [14], and failed state runs should be saved to a DLQ for investigation [1].

6 Performance Efficiency and Caching Strategy

PostgreSQL Full-Text Search Optimization. The tsvector search targeting sub-300ms p95 [4] can be dramatically improved with proper GIN indexing. AWS benchmarks demonstrate that GIN indexes on tsvector columns reduce query time from approximately 5,882ms to about 1.14ms on a 5-million-row table — a roughly 5,000x improvement [9]. GIN indexes use bitmap index scans and are preferred over GiST for read-heavy full-text search workloads [15]. For the scripture search use case, expression-based GIN indexes should be created with explicit text search configurations across book name, chapter, and verse text columns for all 7 translations. The `fastupdate` parameter with `gin_pending_list_limit` tuning can batch index updates for write-heavy periods such as bulk lectionary data imports [9].

RDS Proxy for Connection Pooling. With 29 Lambda functions potentially opening concurrent database connections, connection exhaustion is a significant risk. Each Lambda invocation establishes its own PostgreSQL connection, and during traffic spikes (e.g., Sunday morning when congregations access readings simultaneously), the connection count can exceed PostgreSQL's `max_connections` limit. RDS Proxy should be deployed to pool and share connections across all Lambda functions [10], with monitoring configured to alert when connection utilization exceeds 80%.

HLS Audio Streaming Architecture. CloudFront's three-tier caching architecture — edge locations, Regional Edge Caches (RECs), and origin — is well-suited for HLS audio delivery [16]. Separate cache behaviors should be configured for `.m3u8` manifest files (short TTL of 1–3 seconds for live-like freshness) and `.ts` audio segments (long TTL of 24+ hours since segments are immutable once generated) [17]. For the AWS Polly fallback path serving non-ESV translations, the new Bidirectional Streaming API [18] enables real-time text-to-speech synthesis with lower latency than the batch synthesis approach. A recommended architecture pattern is Polly → S3 → CloudFront for pre-generated audio, with Redis caching for frequently requested passages [19].

Redis Caching Strategy. The current 24-hour TTL for verse caching [4] is appropriate for scripture text that changes infrequently. ElastiCache Serverless should replace self-managed Redis to gain automatic scaling (supporting up to 65,000 concurrent connections per node) and sub-minute provisioning [20]. Lambda functions must be configured within the same VPC as ElastiCache, using IAM-based authentication [21]. Cache key design should incorporate translation, book, chapter, and verse to enable granular invalidation, and expensive operations like KEYS should be replaced with SCAN [22].

7 Cost Optimization and Resource Utilization

Lambda Right-Sizing. Migrating all 29 Lambda functions to Graviton2 (arm64) runtimes delivers approximately 20% cost savings with equivalent or better performance. AWS Lambda Power Tuning should be used to right-size memory allocations per function — scripture search functions may need more memory for tsvector processing, while simple CRUD endpoints for bookmarks can run with minimal allocations.

Reserved Database Capacity. The PostgreSQL instance supporting 8 tables with JSONB columns and GIN indexes

should use Reserved Instances for predictable baseline capacity. The `lectionary_readings` table, which stores readings for every date across multiple lectionary systems, will grow linearly and predictably, making it an ideal candidate for capacity planning.

HLS Cache Optimization. Splitting CloudFront cache behaviors between manifests and segments [17] reduces origin fetches for immutable audio segments. For Polly-generated audio, pre-generating and caching the most commonly accessed passages (e.g., Sunday readings for the upcoming 4 weeks) in S3 eliminates per-request Polly synthesis costs.

Bible API Cost Management. The ESV API's 5,000 requests/day free tier [4] can be stretched significantly with aggressive Redis caching. Since the lectionary assigns specific readings to specific dates, the entire upcoming week's readings can be pre-fetched and cached during off-peak hours, reducing real-time API calls to only ad-hoc scripture searches and reading plan passages.

8 Sustainability and Data Integrity

Efficient Compute Utilization. The serverless-first architecture inherently maximizes utilization by provisioning resources only when work needs to be done [23]. Right-sizing Lambda memory across all 29 functions ensures that no function over-provisions compute capacity. Functions that perform simple database lookups (e.g., bookmark retrieval) should be tuned to the minimum viable memory allocation.

Data Lifecycle Policies. The `lectionary_readings` table accumulates historical readings that are rarely accessed after their liturgical date passes. Implementing a data lifecycle policy that archives readings older than 3 liturgical cycles (9 years) to S3 Glacier reduces active database storage. Similarly, `reading_plan_progress` records for completed or abandoned plans should be archived after a configurable retention period.

Computus Algorithm Correctness. The Anonymous Gregorian Method implementation is validated for 2000–2100 [4], but no automated test suite verifies outputs against known Easter dates. A comprehensive validation suite should test all years in the valid range against authoritative Easter date tables, with particular attention to edge cases where Easter falls at its earliest possible date (March 22) or latest (April 25). The algorithm should also include a runtime guard that rejects years outside the validated range with a clear error message rather than returning potentially incorrect dates.

Lectionary Data Accuracy. Maintaining accurate lectionary data across 5 systems [4] — each with its own reading assignments, seasonal variations, and special observances — requires a validation pipeline. The Roman Catholic system's additional weekday 2-year I/II cycle adds complexity beyond the standard 3-year Sunday cycle. Automated reconciliation against authoritative lectionary sources should run quarterly, with discrepancy alerts routed to designated liturgical administrators.

9 Third-Party Integration and API Resilience

The Scripture and Lectionary flow has the highest third-party API dependency of any ROCA flow, relying on ESV API, Bible.api, and potentially AWS Polly for core functionality. A layered resilience strategy is essential.

Circuit Breaker Implementation. AWS provides multiple reference architectures for the circuit breaker pattern: Step Functions with DynamoDB for state tracking [12], Lambda extensions with DynamoDB [24], and Lambda Powertools' built-in circuit breaker utility [8]. For the Bible API use case, the Lambda Powertools approach is recommended for its simplicity — it can be added as a decorator to existing scripture-fetching functions without introducing new infrastructure. The circuit breaker should maintain separate state per API provider (ESV vs. Bible.api) so that a failure in one does not unnecessarily block the other.

Graceful Degradation Chain. The Well-Architected Framework (REL05-BP01) recommends transforming hard dependencies into soft ones through graceful degradation [25]. The scripture delivery chain should follow a clear fallback hierarchy: (1) Redis cache hit → serve immediately; (2) ESV API → fetch, cache, and serve; (3) Bible.api → fetch, cache, and serve; (4) Local KJV from SQLite/IndexedDB → serve with translation disclaimer. Each fallback should be transparent to the user through UI indicators showing the active translation source.

Retry Strategy. The Well-Architected Framework (REL05-BP03) prescribes exponential backoff with jitter and capped maximum retries [26]. For the ESV API's daily quota limit, retries should be capped at 3 attempts with a maximum backoff of 5 seconds, after which the circuit breaker opens and traffic routes to the fallback chain. The Redis cache-aside strategy with volatile-lru eviction [27] ensures that the most recently accessed verses remain available even when the cache approaches capacity.

10 Batch 4 Prioritized Remediation Roadmap

The following roadmap addresses findings specific to the Scripture and Lectionary Management flow, estimated at 30–40 engineering days of incremental effort.

P0 — Critical (Days 1–14, ~14 engineering days). DLQs must be configured on all async Lambda invocations across the 29 endpoints, with Maximum Retry Attempts and Maximum Record Age settings [1] (~3 days). The Bible API circuit breaker should be implemented using Lambda Powertools [8] with DynamoDB state tracking, covering both ESV API and Bible.api independently (~5 days). WAF WebACLs with AWS Managed Rules must be attached to the API Gateway stage, including rate-limiting rules for the scripture search and devotional generation endpoints [2] (~3 days). GuardDuty must be enabled for Lambda and API Gateway threat monitoring (~1 day). JWT rotation via Cognito with PKCE must be implemented platform-wide (~2 days, leveraging patterns from Batch 1 remediation).

P1 — High (Days 15–30, ~14 engineering days). The offline sync architecture should be upgraded from last-write-wins to AWS AppSync with Optimistic Concurrency conflict resolution [6], integrated with Amplify DataStore for automatic offline support and Delta Sync for efficient pre-caching [13, 7] (~8 days). GIN indexes must be created on all tsvector columns with expression-based configurations per translation, and fastupdate tuning applied [9] (~2 days). RDS Proxy should be deployed for connection pooling across all 29 Lambda functions [10] (~2 days). Bible API keys should be migrated to Secrets Manager with 90-day rotation (~2 days).

P2 — Medium (Days 31–45, ~12 engineering days). The Computus algorithm validation suite should be built with test coverage for all years 2000–2100 and runtime guards for out-of-range inputs [4] (~3 days). EventBridge integration should decouple lectionary-cycle events, reading plan triggers, and scripture-delivery notifications into a dedicated event bus with schema validation [28] (~4 days). The HLS audio architecture should be optimized with separate CloudFront cache behaviors and Polly pre-generation for upcoming Sunday readings [16, 17, 18] (~3 days). X-Ray distributed tracing should be enabled across all endpoints [11] (~2 days).

Priority	Finding	Effort	Dependencies
P0	DLQs on all async Lambdas	~3 days	None
P0	Bible API circuit breaker (Lambda Powertools + DynamoDB)	~5 days	None
P0	WAF + GuardDuty deployment	~4 days	None
P0	JWT rotation with PKCE	~2 days	Cognito setup from B1
P1	AppSync offline sync upgrade	~8 days	Schema design
P1	GIN index optimization	~2 days	None
P1	RDS Proxy deployment	~2 days	VPC configuration
P1	Secrets Manager for API keys	~2 days	None
P2	Computus validation suite	~3 days	None
P2	EventBridge integration	~4 days	Event schema design
P2	HLS audio optimization	~3 days	CloudFront config
P2	X-Ray tracing	~2 days	None

Table: Batch 4 Remediation Roadmap with Effort Estimates and Dependencies

11 Consolidated Cross-Batch Summary: All 8 ROCA Flows (Batches 1–4)

Across four batches covering all 8 ROCA flows — Member Onboarding (B1), Event Discovery and Registration (B2), Event Management and Payments (B2), Volunteer Scheduling (B3), Facility Management (B3), Notification Orchestration (B3), Communication Workflows (B3), and Scripture and Lectionary Management (B4) — the review has identified a set of systemic architectural gaps that compound in severity with each successive batch.

Chart Explanation: This chart shows the estimated engineering effort for each batch’ s remediation roadmap, broken down by priority tier. Batch 1 established the baseline with a 90-day roadmap covering 15 findings [29]. Batch 2 identified 21 gaps requiring 40–55 engineering days [30]. Batch 3 totaled 80–95 engineering days across P0 (~27 days), P1 (~26 days), and P2 (~24 days) [31]. Batch 4 adds approximately 30–40 days for Bible API resilience, offline sync upgrades, search optimization, and audio architecture improvements. The cumulative platform-wide

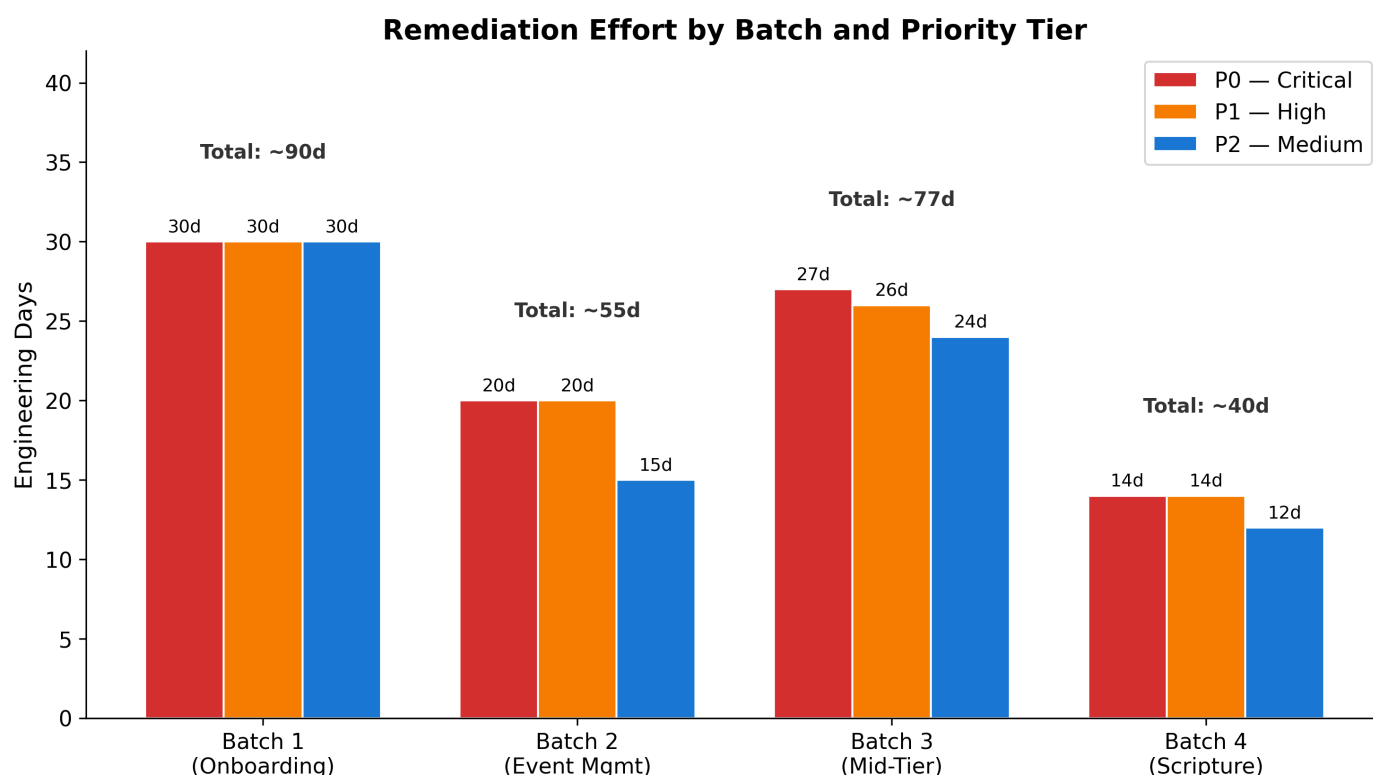


Figure 1: Remediation Effort by Batch

remediation effort is estimated at 240–280 engineering days.

Six Persistent Critical Gaps. The following gaps have appeared in every batch, with severity escalating as new flows exposed additional attack surfaces and failure modes [3]:

Chart Explanation: This heatmap visualizes how eight critical findings have evolved across the four review batches. The darkest cells (CRIT) indicate Critical severity, orange indicates High, yellow indicates Medium, and gray indicates the finding was not applicable to that batch’s flows. JWT rotation, missing DLQs, and WAF/GuardDuty gaps have maintained Critical severity across all applicable batches [3], while WAF/GuardDuty escalated from High in Batch 1 to Critical in Batch 2 when payment processing flows were assessed [3].

Embedded State Machines remain in application code rather than AWS Step Functions across onboarding workflows, event registration, volunteer scheduling, and reading plan orchestration. Step Functions would provide built-in retry logic, visual debugging, and saga-pattern support with compensating transactions [14, 1]. This finding was Critical in Batch 1 and remains High across subsequent batches [29].

JWT Rotation and PKCE have been absent since Batch 1 and represent the single most persistent security vulnerability across the platform. Every API endpoint across all 8 flows is affected. This was identified as Critical in Batch 1 and has maintained that severity through all four reviews [3].

Missing DLQs on async Lambda invocations mean that failed events across onboarding, event management, notifications, and scripture delivery are silently dropped. The Well-Architected Serverless Lens requires DLQs on every async invocation [1], making this a compliance gap as well as a reliability risk.

WAF and GuardDuty Absence leaves all API Gateway endpoints unprotected against common web exploits [2]. This gap escalated from High to Critical in Batch 2 when Stripe payment endpoints were assessed, and remains Critical through Batch 4 where public-facing scripture search endpoints accept arbitrary user input [3].

EventBridge Integration remains at High severity as a P2 item. Without a centralized event bus, cross-flow communication (e.g., lectionary cycle changes triggering notification updates, event cancellations cascading to volunteer schedules) relies on direct service-to-service calls, creating tight coupling [28].

Flow-Specific Critical Findings. Beyond the persistent gaps, each batch introduced flow-specific Critical findings: RSVP race conditions with lost-update anomalies in Batch 2 [3], PCI DSS v4.0 compliance violations for Stripe integration in Batch 2 [30], SES bounce/complaint handling absence in Batch 3 [31], FCM HTTP v1 migration overdue in Batch 3 [31], and Bible API resilience gaps in Batch 4 [4].

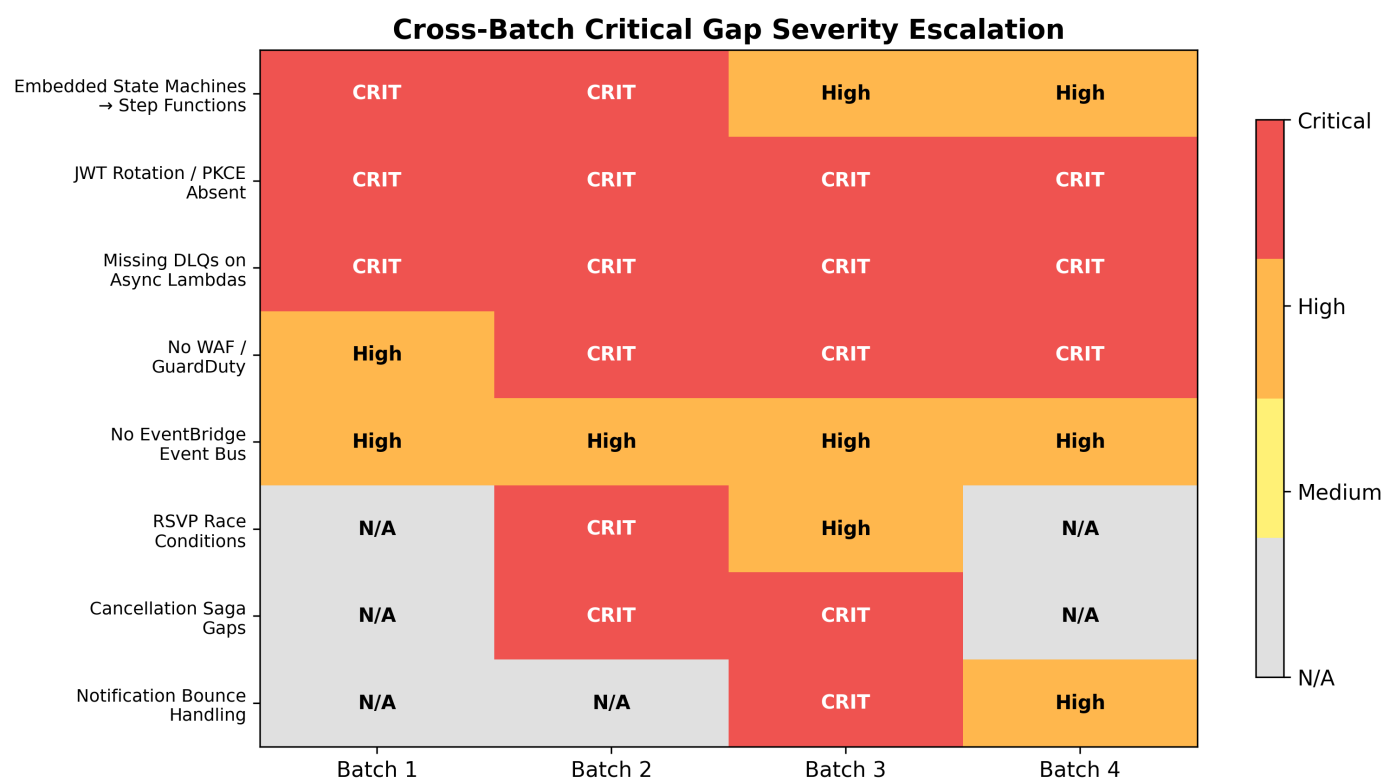


Figure 2: Cross-Batch Severity Heatmap

Recommended Platform-Wide Remediation Sequence. Given the cumulative 240–280 engineering days of remediation across all batches, the following sequencing is recommended. Phase 1 (Weeks 1–4) should address the three gaps that affect every flow simultaneously: WAF/GuardDuty deployment, DLQ configuration on all async Lambdas, and JWT rotation with PKCE — these are force-multiplier fixes that reduce risk across all 8 flows at once. Phase 2 (Weeks 5–10) should tackle flow-specific Critical items: PCI DSS remediation for payments, SES bounce handling for notifications, Bible API circuit breakers for scripture delivery, and RSVP concurrency controls for event registration. Phase 3 (Weeks 11–16) should implement architectural improvements: Step Functions migration for embedded state machines, AppSync for offline sync, EventBridge for cross-flow decoupling, and GIN index optimization. Phase 4 (Weeks 17–20) should complete optimization items: Graviton2 migration, Lambda right-sizing, HLS cache tuning, Computus validation, and X-Ray observability. This sequencing ensures that the highest-risk, broadest-impact items are resolved first while building the architectural foundations needed for later optimizations.

12 References

- <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/failure-management.html>
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>
- AWS Well-Architected Framework Review for ROCA Event Management Flows with Batch 1 Gap Analysis
- ROCA_Scripture_Lectinary_Design_v1.0.docx
- <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>
- <https://docs.aws.amazon.com/appsync/latest/devguide/conflict-detection-and-resolution.html>
- <https://docs.aws.amazon.com/appsync/latest/devguide/tutorial-delta-sync.html>
- https://docs.aws.amazon.com/powertools/python/latest/utilities/circuit_breaker/
- <https://aws.amazon.com/blogs/database/index-types-supported-in-amazon-aurora-postgresql-and-amazon-rds-for-postgresql-gin-gist-hash-brin>
- <https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>
- <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>

12. <https://aws.amazon.com/blogs/compute/using-the-circuit-breaker-pattern-with-aws-step-functions-and-amazon-dynamodb/>
13. <https://www.serverless.com/guides/aws-appsync>
14. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
15. <https://www.postgresql.org/docs/17/textsearch-indexes.html>
16. <https://docs.aws.amazon.com/whitepapers/latest/amazon-cloudfront-media/architecture.html>
17. <https://docs.aws.amazon.com/whitepapers/latest/amazon-cloudfront-media/architecture-of-media-workflows.html>
18. <https://aws.amazon.com/about-aws/whats-new/2026/03/amazon-polly-expands-TTS-new-voices-and-bidirectional-streaming/>
19. <https://github.com/West-Michigan-AWS-Users-Group/CloudChirp>
20. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/BestPractices.Clients.Redis.Connections.html>
21. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/LambdaRedis.html>
22. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WorkingWithRedis.html>
23. <https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/design-principles-for-sustainability-in-the-cloud.html>
24. <https://aws.amazon.com/blogs/compute/using-the-circuit-breaker-pattern-with-aws-lambda-extensions-and-amazon-dynamodb/>
25. https://docs.aws.amazon.com/wellarchitected/2022-03-31/framework/rel_mitigate_interaction_failure_graceful_deg
26. https://docs.aws.amazon.com/wellarchitected/2024-06-27/framework/rel_mitigate_interaction_failure_limit_retries
27. <https://docs.aws.amazon.com/whitepapers/latest/database-caching-strategies-using-redis/evictions.html>
28. https://docs.aws.amazon.com/en_us/eventbridge/latest/userguide/eb-rule-dlq.html
29. AWS Well-Architected Framework Review of ROCA New Member Onboarding Flow
30. AWS Well-Architected Framework Review for ROCA Event Management Flows: Payment Security, Concurrency, and Operational Resilience Assessment
31. AWS Well-Architected Framework Review for ROCA Middle-Tier Flows: Orchestration, Scheduling, and Notification Architecture